EXAMEN UD2 – RA3 (7 noviembre 2025)

Escribe y depura código, analizando y utilizando las estructuras de control del
Duración: 2 horas **Lenguaje:** Java **Temática:** Halloween 🎃

Criterios de evaluación RA3

a) Se ha escrito y probado código que haga uso de estructuras de selección.
b) Se han utilizado estructuras de repetición.
c) Se han reconocido las posibilidades de las sentencias de salto.
e) Se han creado programas ejecutables utilizando diferentes estructuras de control.
f) Se han probado y depurado los programas.
g) Se ha comentado y documentado el código.

Indicaciones

- Todo mediante **métodos**; en `main` solo pruebas.
- Usa **`Integer`** y **`List<Integer>`**.
- Emplea las estructuras exigidas en cada enunciado.
- Documenta las decisiones.

🕯️ Ejercicio 1 — Control de acceso al Baile de los Monstruos

En el gran baile de Halloween solo pueden entrar:

- Quienes tengan al menos **18 años** o estén **acompañados**.
- Los **vampiros registrados** tienen **entrada libre**.

Crea un método que reciba la edad, si está acompañado y el tipo de criatura,
y devuelva `true` si puede entrar o `false` si no.

```
public static boolean puedeEntrar(Integer edad, boolean
acompanado, String criatura) {

    // TODO

}
```

Ejemplo de uso:

```
System.out.println(puedeEntrar(16, true, "zombie"));   // true

System.out.println(puedeEntrar(15, false, "vampiro")); // true

System.out.println(puedeEntrar(15, false, "bruja"));   //false
```

Criterios: a, e, f, g

🧛 Ejercicio 2 — Clasificador de caramelos

Los caramelos recolectados tienen etiquetas: `"dulce"`, `"ácido"`, `"salado"`, `"misterioso"`.
Crea un método que reciba una lista y cuente cuántos hay de un tipo dado, ignorando nulos.

```
public static Integer cuentaCaramelos(List<String> caramelos,
String tipo) {

    // TODO

}
```

Ejemplo de uso:

```
List<String> lista = List.of("dulce", "ácido", "dulce",
"misterioso", null);
```

```
System.out.println(cuentaCaramelos(lista, “dulce”));

// 2
```

Criterios: a, b, c, e, f, g

🕸️ Ejercicio 3 — Contador de pasos en el Laberinto

(while + break)
Un monstruo recorre un laberinto representado por una lista de enteros.
Cada número indica la distancia recorrida.
Cuando el número es negativo, ha caído en una trampa y el recorrido termina.
Devuelve el total de pasos **antes de la trampa** (no se suma el negativo).

```
public static Integer pasosLaberinto(List<Integer> pasos) {

    // TODO

}
```

Ejemplo de uso:

```
List<Integer> recorrido = List.of(2, 3, 4, -1, 5);

System.out.println(pasosLaberinto(recorrido)); // 9
```

Criterios: b, c, e, f, g

🎃 Ejercicio 4 — Nivel medio de magia en las calabazas

Cada calabaza tiene un nivel de magia (0–10).
Crea un método que calcule el **promedio de los valores mayores o iguales que 5**, ignorando los inferiores.

```
public static Double promedioMagia(List<Integer> niveles) {

    // TODO

}
```

Ejemplo de uso:

```
List<Integer> magia = List.of(3, 9, 7, 4, 10);

System.out.println(promedioMagia(magia)); // (9+7+10)/3 = 8.67
```

Criterios: a, b, e, f, g

🧟 Ejercicio 5 — Recursividad: Cuenta Atrás de Sustos

Crea un método recursivo que simule una cuenta atrás desde un número hasta 0, imprimiendo cada número con un mensaje tipo “¡Boo n!”.
Cuando llegue a 0, imprime “¡Susto final!”.

```
public static void cuentaAtrasSustos(Integer n) {

    //

}
```

Ejemplo de uso:

```
cuentaAtrasSustos(4);

// Boo 4

// Boo 3

// Boo 2

// Boo 1

// ¡Susto final!
```

Criterios: a, c, e, f, g

Ejercicio 6 — La suma entre los dos ceros

Hacer un método que reciba una lista de enteros de la cual conocemos que tiene dos ceros. Calcula la suma de los números que hay entre esos dos ceros .

```
public static Integer sumaEntreCeros(List<Integer> numeros) {

    // TODO

}
```

Ejemplo de uso:

```
List<Integer> numeros = List.of(3, 9, 0, 4, 10, 0, -3, 5);

System.out.println(sumaEntreCeros(magia)); // 4+10 = 14
```

Criterios: a, b, c, e, f, g